Documentación Final TPO — Sistemas Operativos (Red Hat)

Pasos iniciales para configurar el repositorio

En primer lugar utilizamos GitHub como herramienta de acceso a los scripts. El repositorio se encuentra en: https://github.com/Lucasalexisc/sistemas-operativos.git

Desde Red Hat nos conectamos a la terminal y ejecutamos: `git clone https://github.com/Lucasalexisc/sistemas-operativos.git`

```
[student@workstation ~]$ git clone https://github.com/Lucasalexisc/sistemas-operativos.git
Cloning into 'sistemas-operativos'...
remote: Enumerating objects: 219, done.
remote: Counting objects: 100% (219/219), done.
remote: Compressing objects: 100% (170/170), done.
remote: Total 219 (delta 66), reused 183 (delta 38), pack-reused 0 (from 0)
Receiving objects: 100% (219/219), 30.13 MiB | 41.42 MiB/s, done.
Resolving deltas: 100% (66/66), done.
[student@workstation ~]$ cd sistemas-operativos/
```

Una vez dentro, ejecutamos `cd sistemas-operativos/` y luego `./preparar_todo.sh`, el cual instala todas las librerías y dependencias necesarias (`gcc`, `strace`, `acl`, `file`, `sha256sum`), otorga permisos de ejecución (`chmod +x`) a todos los archivos `.sh` y deja el entorno preparado:

Primero otorga permisos `chmod +x` a todos los scripts `.sh`:

```
================================================================================
 1. CONFIGURANDO PERMISOS DE EJECUCION (+x) POR CADA UNIDAD
================================================================================
 -> [Unidad 1]: Otorgando chmod +x a 'audit_syscalls.sh' y 'sync_service.sh' (Syscalls y Kernel)...
 -> [Unidad 2]: Otorgando chmod +x a 'demostracion_tpo.sh' y 'carrera_escritura.sh' (Concurrencia y Mutex)...
 -> [Unidad 3]: Otorgando chmod +x a 'ejecutar_tpo.sh' (Memoria RAM: First-Fit y Best-Fit)...
 -> [Unidad 4]: Otorgando chmod +x a 'ejecutar_tpo.sh' (File System: Asignacion Contigua vs Inodos)...
 -> [Unidad 5]: Otorgando chmod +x a 'ejecutar_tpo.sh' (Entrada/Salida: Sin Bufer vs Con Bufer)...
 -> [Unidad 6]: Otorgando chmod +x a scripts de Seguridad, Firewall y Hash...
 -> [Unidad 6]: Aplicando chmod 750 restringido a 'pagos_empresa.sh' (Demostracion de permisos y ACLs)...
 -> [General ]: Otorgando chmod +x a 'menu_tpo.sh' y 'preparar_todo.sh'...
[OK]: Todos los scripts de las Unidades 1 a 6 tienen sus permisos listos.
      (Nota: Los archivos .c no requieren chmod +x ya que los compila gcc directamente).
```

Luego verifica e instala automáticamente las herramientas necesarias como `gcc`:

```
================================================================================
 2. DIAGNOSTICO Y REPARACION DE REPOSITORIOS EN RED HAT
================================================================================
 -> [OK]: Los repositorios de Red Hat estan limpios y sin conflictos.

================================================================================
 3. VERIFICACION DE HERRAMIENTAS Y LIBRERIAS POR UNIDAD
================================================================================
  [FALTA] 'gcc' (Compilador C) NO instalado -> Requerido en Unidades 2, 3, 4 y 5.
  [OK] 'strace' (Auditor Syscalls) ya instalado -> Utilizado en Unidades 1 y 5.
  [OK] 'getfacl' (Paquete acl) ya instalado -> Utilizado en Unidad 6 (Seguridad y ACLs).
  [OK] 'file' (Numeros magicos) ya instalado -> Utilizado en Unidad 4 (File System e Inodos).
  [OK] 'sha256sum' ya instalado -> Utilizado en Unidad 6 (Comprobador Hash de Integridad).

================================================================================
 4. INSTALACION AUTOMATICA DE DEPENDENCIAS FALTANTES CON SUDO DNF
================================================================================
Instalando paquetes: gcc
(Si el sistema solicita password de sudo, ingresa: student o redhat)

[sudo] password for student:
Red Hat Enterprise Linux 9.3 BaseOS (dvd)       347 kB/s | 2.7 kB     00:00
Red Hat Enterprise Linux 9.3 AppStream (dvd)    1.2 MB/s | 3.2 kB     00:00
Dependencies resolved.
================================================================================
 Package        Arch   Version         Repository                        Size
================================================================================
Installing:
 gcc            x86_64 11.4.1-2.1.el9  rhel-9.3-for-x86_64-appstream-rpms  32 M
Installing dependencies:
 glibc-devel    x86_64 2.34-83.el9_3.7 rhel-9.3-for-x86_64-appstream-rpms  56 k
```

Una vez listo, nos indica las dos formas de ejecución disponibles (`cd unidad_X` o `./menu_tpo.sh`):

```
================================================================================
 [LISTO]: ENTORNO PREPARADO EXITOSAMENTE
 Podes ejecutar cualquier ejercicio:
   a) Ingresando a la unidad (ej: cd unidad_3 && ./ejecutar_tpo.sh)
   b) O ejecutando el menu general interactivo: ./menu_tpo.sh
================================================================================
```

Unidad 1 — Llamadas al Sistema (Syscalls) y Auditoría con strace

Introducción breve

En la empresa **SecurePay FinTech Solutions**, cada medianoche se ejecuta en un servidor Red Hat el script `sync_service.sh`, encargado de conciliar las transacciones del día. Al pasar a producción, el proceso empezó a cortarse a los pocos segundos mostrando únicamente el mensaje `[ERROR FATAL 101]: Fallo critico en el motor transaccional`. El mensaje no indica qué falló, y las áreas se culpaban entre sí (desarrollo sospechaba de la base de datos y el proveedor decía que el problema era el servidor). El objetivo es encontrar la causa real **sin modificar el código del servicio**, observando qué le pide el programa al Kernel.

Temas expuestos

- **Modo Usuario y Modo Kernel:** los programas comunes (Modo Usuario) no pueden acceder directamente al disco ni al hardware; tienen que pedírselo al Kernel, que es el único que trabaja con privilegios completos.
- **Llamadas al sistema (syscalls):** son la forma en que un programa le pide un servicio al Kernel (abrir un archivo, leerlo, escribir en pantalla). Si la operación falla, el Kernel devuelve `-1` y un código de error (por ejemplo `ENOENT` = el archivo no existe, `EACCES` = permiso denegado).
- **`strace`:** herramienta de Linux que intercepta y muestra todas las llamadas al sistema que hace un programa, con su resultado, sin necesidad de tocar su código.
- **Descriptores de archivo:** `0` entrada estándar, `1` salida estándar y `2` salida de errores (por eso el mensaje de error usa `>&2`).

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./sync_service.sh`	El servicio falla con `[ERROR FATAL 101]` (el problema).
2	`./audit_syscalls.sh`	`strace` revela la causa: `faccessat2(... "/etc/secure_app/transactions.dat" ...) = -1 ENOENT`.
3	`./reparar_servicio.sh`	Se crea la carpeta y el archivo faltante; el servicio termina con código `0` y `strace` muestra `= 0`.
4 (opcional)	`./restaurar_falla.sh`	Vuelve a dejar el servidor con el problema para repetir la demostración.

El comando `chmod` solo funciona sobre archivos que **ya existen**. Por eso la solución no es solamente `chmod 644`: primero hay que crear la carpeta `/etc/secure_app` y el archivo `transactions.dat`, que es lo que hace `reparar_servicio.sh`.

Código sync_service.sh (el servicio que falla)

unidad_1/sync_service.sh

```
#!/bin/bash
# ============================================================================
# UNIDAD 1 - SERVICIO BANCARIO DE CONCILIACION (sync_service.sh)
# Este script simula el programa de la empresa que falla con 'ERROR FATAL 101'
# sin indicar la causa real. Lo auditaremos desde afuera con 'strace'.
# ============================================================================

# 1. Guardamos en la variable DATA_FILE la ruta del archivo de transacciones
DATA_FILE="/etc/secure_app/transactions.dat"

# 2. Mostramos en pantalla (stdout - descriptor 1) que el servicio comienza
echo "[INFO] Iniciando el servicio de conciliacion..."

# 3. Verificamos si el archivo NO se puede leer:
#    -r comprueba si el archivo existe y tiene permisos de lectura (invoca la syscall faccessat2).
#    !  niega la condicion (entra al if si NO existe o NO tiene permisos).
if [ ! -r "$DATA_FILE" ]; then
    # >&2 envia el mensaje por el canal estandar de errores (stderr - descriptor 2)
    echo "[ERROR FATAL 101]: Fallo critico en el motor transaccional. Contacte a soporte de IT." >&2
    # Cortamos la ejecucion devolviendo el codigo de error 101 (distinto de 0 = fallo)
    exit 101
fi

# 4. Si el archivo existe y tiene permisos de lectura, mostramos su contenido con 'cat'
cat "$DATA_FILE"

# 5. Finalizamos devolviendo 0 (convencion en Linux para indicar ejecucion exitosa)
exit 0
```

Desglose línea por línea

- **`#!/bin/bash`**: indica al sistema operativo que el archivo debe ejecutarse con el intérprete Bash.
- **`DATA_FILE="/etc/secure_app/transactions.dat"`**: guarda en la variable `DATA_FILE` la ruta del archivo de transacciones que el servicio necesita leer.
- **`echo "[INFO] Iniciando el servicio de conciliacion..."`**: muestra en pantalla un mensaje informativo indicando que el servicio comenzó.
- **`if [ ! -r "$DATA_FILE" ]; then`**: antes de continuar, verifica una condición. `-r` comprueba si el archivo existe y se puede leer; el `!` niega esa condición. En conjunto significa: *"si el archivo NO se puede leer"* (porque no existe o porque no hay permiso de lectura).
- **`echo "[ERROR FATAL 101]: ..." >&2`**: si no se puede leer, muestra el mensaje de error. `>&2` envía el texto por la salida de errores (descriptor 2).
- **`exit 101`**: termina el script en ese momento devolviendo el código `101`, que indica que finalizó con error.
- **`fi`**: cierra el bloque `if`.
- **`cat "$DATA_FILE"`**: si el archivo sí se pudo leer, muestra su contenido en pantalla.
- **`exit 0`**: termina el script devolviendo `0`, que en Linux significa que finalizó correctamente.

En resumen

El script guarda la ruta del archivo de transacciones, avisa que está iniciando y verifica si puede leer ese archivo. Si no puede, muestra el Error Fatal 101 y se detiene con código `101`. Si puede, muestra las transacciones y termina con código `0`. El problema es que el mensaje de error no dice **por qué** no pudo leer el archivo.

Código audit_syscalls.sh (el diagnóstico)

unidad_1/audit_syscalls.sh

```
#!/bin/bash
# ================================================================================
# UNIDAD 1 - AUDITORIA DE LLAMADAS AL SISTEMA CON STRACE (audit_syscalls.sh)
# Objetivo: Descubrir por que falla 'sync_service.sh' sin modificar su codigo,
# interceptando la comunicacion entre Modo Usuario (Ring 3) y Modo Kernel (Ring 0).
# ================================================================================

# 0. Verificamos que la herramienta 'strace' este instalada en Red Hat
#    command -v busca el binario; >/dev/null 2>&1 oculta la salida de verificacion
if ! command -v strace >/dev/null 2>&1; then
    echo "============================================================"
    echo " [ERROR]: 'strace' no esta instalado en este sistema Red Hat."
    echo " Para instalarlo automaticamente junto a las demas herramientas:"
    echo "   cd .. && ./preparar_todo.sh"
    echo " O manualmente: sudo dnf install -y --disablerepo='*ansible*' strace"
    echo "============================================================"
    exit 1
fi

# Otorgamos permiso de ejecucion (+x) al script que vamos a auditar
chmod +x ./sync_service.sh

echo "============================================================"
echo " 1. INTERCEPTANDO SYSCALLS CON STRACE"
echo "============================================================"
# Ejecutamos sync_service.sh monitoreado por strace:
#   -f               : sigue tambien a los procesos hijos creados por el script
#   -e trace=file    : filtra solo llamadas al sistema que tocan archivos (openat, faccessat2, stat)
#   -o auditoria...  : guarda el registro completo en un archivo de texto en vez de saturar la
pantalla
strace -f -e trace=file -o auditoria_archivos.log ./sync_service.sh

echo ""
echo "============================================================"
echo " 2. DIAGNOSTICO: CAUSA RAIZ DETECTADA A NIVEL KERNEL"
echo "============================================================"
# Usamos 'grep' para buscar dentro del log unicamente la linea referida a 'transactions.dat'.
# Aqui se ve que la syscall faccessat2 devuelve '-1 ENOENT (No such file or directory)'.
grep "transactions.dat" auditoria_archivos.log

echo ""
echo "============================================================"
echo " 3. TABLA ESTADISTICA DE LLAMADAS AL SISTEMA (-c)"
echo "============================================================"
```

```
# Ejecutamos strace con '-c' (count) para armar una tabla resumen que muestra:
# cantidad de llamadas (calls), errores (errors) y tiempo de CPU consumido en Modo Kernel.
strace -c -e trace=execve,openat,read,write,close,newfstatat,faccessat2 ./sync_service.sh
```

Salida en consola al ejecutar ./audit_syscalls.sh:

```
============================================================
 1. INTERCEPTANDO SYSCALLS CON STRACE
============================================================
[INFO] Iniciando el servicio de conciliacion...
[ERROR FATAL 101]: Fallo critico en el motor transaccional. Contacte a soporte de IT.

============================================================
 2. DIAGNOSTICO: CAUSA RAIZ DETECTADA A NIVEL KERNEL
============================================================
3007  faccessat2(AT_FDCWD, "/etc/secure_app/transactions.dat", R_OK, AT_EACCESS) = -1 ENOENT (No such file or directory)

============================================================
 3. TABLA ESTADISTICA DE LLAMADAS AL SISTEMA (-c)
============================================================
[INFO] Iniciando el servicio de conciliacion...
[ERROR FATAL 101]: Fallo critico en el motor transaccional. Contacte a soporte de IT.
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         6           read
  0.00    0.000000           0         2           write
  0.00    0.000000           0        22           close
  0.00    0.000000           0         1           execve
  0.00    0.000000           0        33        12 openat
  0.00    0.000000           0        29           newfstatat
  0.00    0.000000           0         1         1 faccessat2
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0        94        13 total
```

Desglose línea por línea

- **`if ! command -v strace >/dev/null 2>&1; then ... exit 1; fi`**: verifica que `strace` esté instalado. Si no lo está, muestra cómo instalarlo y termina con `exit 1`. La parte `>/dev/null 2>&1` hace que la verificación no imprima nada en pantalla.
- **`chmod +x ./sync_service.sh`**: le da permiso de ejecución al servicio para poder correrlo.
- **`strace -f -e trace=file -o auditoria_archivos.log ./sync_service.sh`**: ejecuta el servicio a través de `strace`:
  - `-f`: también registra los procesos hijos que cree el script.
  - `-e trace=file`: registra solamente las llamadas al sistema relacionadas con archivos.
  - `-o auditoria_archivos.log`: guarda todo el registro en ese archivo en lugar de mostrarlo en pantalla.
- **`grep "transactions.dat" auditoria_archivos.log`**: busca dentro del registro únicamente las líneas que mencionan `transactions.dat`. Así aparece en pantalla la llamada exacta y la respuesta del Kernel.
- **`strace -c -e trace=execve,openat,read,write,close,newfstatat,faccessat2 ./sync_service.sh`**: vuelve a ejecutar el servicio con `-c`, que en lugar de mostrar cada llamada arma una tabla con cuántas veces se usó cada una, cuántas fallaron y cuánto tiempo consumieron.

En resumen

Este script corre el servicio "vigilado" por `strace`, guarda todo lo que el servicio le pidió al Kernel sobre archivos, filtra la línea de `transactions.dat` para mostrar el error real y, por último, presenta una tabla estadística de las llamadas al sistema. La línea clave que aparece es:

```
faccessat2(AT_FDCWD, "/etc/secure_app/transactions.dat", R_OK, AT_EACCESS) = -1 ENOENT (No such file or directory)
```

Es decir, el Kernel responde que **el archivo no existe** (`ENOENT`).

Código reparar_servicio.sh (la solución)

unidad_1/reparar_servicio.sh

```
#!/bin/bash
# ================================================================================
# SOLUCION - UNIDAD 1 (reparar_servicio.sh)
# Causa raiz detectada con strace: faccessat2(".../transactions.dat") = -1 ENOENT
# Como ni la carpeta '/etc/secure_app' ni el archivo 'transactions.dat' existen,
# este script los crea, asigna permisos de lectura y verifica que el servicio funcione.
# ================================================================================

echo "--- 1. CREANDO EL DIRECTORIO /etc/secure_app ---"
# Usamos sudo porque '/etc' es un directorio del sistema protegido por root.
# El parametro -p crea el directorio sin dar error si ya existia previamente.
sudo mkdir -p /etc/secure_app

echo "--- 2. CREANDO EL ARCHIVO transactions.dat CON DATOS DE PRUEBA ---"
# Cargamos 3 transacciones dentro de /etc/secure_app/transactions.dat usando un bloque EOF
sudo bash -c 'cat << EOF > /etc/secure_app/transactions.dat
TXN_ID: 10001 | MONTO: 15420.50 | ESTADO: APROBADO
TXN_ID: 10002 | MONTO: 8930.00  | ESTADO: APROBADO
TXN_ID: 10003 | MONTO: 42100.75 | ESTADO: APROBADO
EOF'

echo "--- 3. ASIGNANDO PERMISOS DE LECTURA (644) ---"
# Ahora que el archivo YA existe en el disco, le damos permisos 644 (rw-r--r--):
# el propietario puede leer/escribir y el resto de los usuarios puede leerlo.
sudo chmod 644 /etc/secure_app/transactions.dat
ls -l /etc/secure_app/

echo "--- 4. VOLVIENDO A EJECUTAR EL SERVICIO ---"
# Ejecutamos nuevamente el servicio; $? muestra el codigo de salida (0 = sin errores)
./sync_service.sh
echo "Codigo de salida: $?"

echo "--- 5. CONFIRMANDO CON STRACE QUE EL KERNEL AHORA RESPONDE 0 (EXITO) ---"
# Repetimos la auditoria con strace para demostrar que faccessat2 ahora retorna '= 0'
strace -f -e trace=file -o auditoria_ok.log ./sync_service.sh > /dev/null
grep "transactions.dat" auditoria_ok.log
```

Salida en consola tras reparar el servicio:

```
============================================================
 3. TABLA ESTADISTICA DE LLAMADAS AL SISTEMA (-c)
============================================================
[INFO] Iniciando el servicio de conciliacion...
TXN_ID: 10001 | MONTO: 15420.50 | ESTADO: APROBADO
TXN_ID: 10002 | MONTO: 8930.00  | ESTADO: APROBADO
TXN_ID: 10003 | MONTO: 42100.75 | ESTADO: APROBADO
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 37.34    0.000230           6        34        12 openat
 37.01    0.000228           5        43         4 newfstatat
 15.58    0.000096           4        22           close
  6.82    0.000042           4         9           read
  1.79    0.000011          11         1           write
  1.46    0.000009           9         1           faccessat2
  0.00    0.000000           0         1           execve
------ ----------- ----------- --------- --------- ----------------
100.00    0.000616           5       111        16 total
```

Desglose línea por línea

- **`sudo mkdir -p /etc/secure_app`**: crea la carpeta que faltaba. Se usa `sudo` porque `/etc` solo puede modificarlo el administrador; `-p` evita errores si la carpeta ya existe.
- **`sudo bash -c 'cat << EOF > /etc/secure_app/transactions.dat ... EOF'`**: crea el archivo `transactions.dat` y escribe dentro tres transacciones de prueba. Todo lo que está entre `<< EOF` y `EOF` es el contenido que se guarda.
- **`sudo chmod 644 /etc/secure_app/transactions.dat`**: asigna permisos `644`: el dueño puede leer y escribir, y el resto de los usuarios solo puede leer (necesario para que el servicio pueda leerlo).
- **`ls -l /etc/secure_app/`**: muestra el archivo creado con sus permisos (`-rw-r--r--`).
- **`./sync_service.sh`** y **`echo "Codigo de salida: $?"`**: vuelve a ejecutar el servicio y muestra su código de salida. `$?` guarda el código del último comando ejecutado; ahora debe ser `0`.
- **`strace -f -e trace=file -o auditoria_ok.log ./sync_service.sh > /dev/null`** y **`grep "transactions.dat" auditoria_ok.log`**: repite la auditoría para confirmar que ahora el Kernel responde `= 0` (éxito) en lugar de `-1 ENOENT`.

En resumen

El script crea la carpeta y el archivo que el servicio no encontraba, le da permisos de lectura, vuelve a ejecutar el servicio (que ahora muestra las transacciones y termina con código `0`) y confirma con `strace` que el Kernel ya no devuelve error.

Código restaurar_falla.sh (para repetir la demostración)

unidad_1/restaurar_falla.sh

```
#!/bin/bash
# ============================================================================
# RESTAURAR FALLA - UNIDAD 1 (restaurar_falla.sh)
# Elimina la carpeta /etc/secure_app para volver a dejar el servidor con la falla
# original (ENOENT) y poder repetir la demostracion en vivo frente al profesor.
# ============================================================================

echo "--- 1. ELIMINANDO /etc/secure_app Y EL ARCHIVO transactions.dat ---"
# Borramos el directorio creado en /etc y limpiamos los archivos de log temporales
sudo rm -rf /etc/secure_app
rm -f auditoria_archivos.log auditoria_ok.log

echo "--- 2. COMPROBANDO QUE EL SERVICIO VUELVE A FALLAR CON ERROR 101 ---"
./sync_service.sh
echo "Codigo de salida: $?"
```

Desglose línea por línea

- **`sudo rm -rf /etc/secure_app`**: borra la carpeta y el archivo creados por la solución, dejando el servidor como estaba al principio.
- **`rm -f auditoria_archivos.log auditoria_ok.log`**: borra los registros de auditorías anteriores.
- **`./sync_service.sh`** y **`echo "Codigo de salida: $?"`**: comprueba que el servicio vuelve a fallar con código `101`.

Cierre de la Unidad 1

Con `audit_syscalls.sh` demostramos que el "fallo crítico en el motor transaccional" no era un error del software ni de la base de datos: al evaluar `[ ! -r "$DATA_FILE" ]`, el programa pasa de Modo Usuario a Modo Kernel mediante la llamada al sistema `faccessat2`, y el Kernel responde `-1 ENOENT`, es decir, que el archivo `/etc/secure_app/transactions.dat` no existe. Lo descubrimos sin modificar el código del servicio, solo observando sus llamadas al sistema con `strace`. Finalmente, con `reparar_servicio.sh` creamos la carpeta y el archivo con permisos `644`, y comprobamos que el servicio finaliza con código `0` y que el Kernel ahora responde `= 0`.

Unidad 2 — Concurrencia, Condición de Carrera y Exclusión Mutua

Introducción breve

En la billetera virtual **Fintech Global Wallet**, el motor de transacciones corre en un servidor Red Hat usando varios hilos en paralelo. Durante un evento de alta demanda se detectaron dos anomalías:

1. **En memoria:** 8 hilos debían acreditar 250.000 depósitos de `$1` cada uno en una misma cuenta (total esperado: 2.000.000), pero el saldo final siempre era menor y cambiaba en cada ejecución (por ejemplo, 439.877).

2. **En un archivo:** dos procesos escribían el estado de una orden en el mismo archivo y uno borraba el dato del otro.

En ningún caso el sistema mostró errores: todos los procesos terminaron con código `0`, pero los datos quedaron corruptos.

Temas expuestos

- **Condición de carrera (*race condition*):** ocurre cuando varios hilos o procesos modifican el mismo dato al mismo tiempo sin coordinarse. El resultado depende de qué hilo llegue primero.
- **Operación no atómica:** `contador_compartido++` parece una sola instrucción, pero el procesador la hace en tres pasos (leer el valor, sumarle 1 y guardarlo). Si dos hilos leen el mismo valor antes de que el otro guarde, una de las sumas se pierde.
- **Sección crítica:** es la parte del código donde se accede al dato compartido.
- **Exclusión mutua (Mutex):** un cerrojo que garantiza que solo un hilo a la vez entre a la sección crítica; los demás esperan su turno.
- **Costo de la sincronización (*overhead*):** el Mutex da resultados correctos pero hace el programa más lento, porque el Kernel interviene para dormir y despertar a los hilos que esperan.

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./demostracion_tpo.sh`	`carrera` da un saldo incorrecto; `seguro` (la solución con Mutex) da 2.000.000; `time` muestra el costo del Mutex.

Código demostracion_tpo.sh

unidad_2/demostracion_tpo.sh

```
#!/bin/bash
# ============================================================================
# UNIDAD 2 - DEMOSTRACION DE CONDICION DE CARRERA VS MUTEX (demostracion_tpo.sh)
# Compila y compara dos programas multi-hilo en C (8 hilos x 250.000 depositos):
#   1) carrera.c : sin sincronizacion (pierde depositos por condicion de carrera)
#   2) seguro.c  : con exclusion mutua pthread_mutex (da siempre 2.000.000 exactos)
# ============================================================================

# 0. Verificamos que el compilador de C (gcc) este instalado en Red Hat
if ! command -v gcc >/dev/null 2>&1; then
    echo "============================================================"
    echo " [ERROR]: 'gcc' no esta instalado en este sistema Red Hat."
    echo " Para instalarlo automaticamente junto a las demas herramientas:"
    echo "   cd .. && ./preparar_todo.sh"
    echo " O manualmente: sudo dnf install -y --disablerepo='*ansible*' gcc"
    echo "============================================================"
    exit 1
fi

echo "=== DEMOSTRACION CONDICION DE CARRERA - UNIDAD 2 ==="
echo "Compilando programas en C..."

# 1. Compilamos ambos programas vinculando la libreria POSIX Threads (-lpthread)
#    que es obligatoria en Linux para crear hilos y usar cerrojos Mutex.
gcc carrera.c -o carrera -lpthread
gcc seguro.c -o seguro -lpthread

# $? guarda el codigo de retorno del ultimo comando; si es distinto de 0 (-ne 0) hubo error
if [ $? -ne 0 ]; then
    echo "Error en la compilacion."
    exit 1
fi

# 2. Ejecutamos el programa sin proteccion: los 8 hilos se pisan al hacer 'contador++'
echo -e "\n1. Ejecutando programa con condicion de carrera (sin sincronizacion):"
./carrera

# 3. Ejecutamos el programa con Mutex: solo 1 hilo a la vez entra a la seccion critica
echo -e "\n2. Ejecutando programa seguro con pthread_mutex (con exclusion mutua):"
./seguro

# 4. Medimos el impacto en tiempos de CPU con el comando 'time':
#    - real: tiempo total de reloj
#    - user: tiempo de CPU en Modo Usuario
#    - sys : tiempo de CPU en Modo Kernel (aumenta con Mutex por la syscall 'futex')
echo -e "\n3. Medicion de tiempos de ejecucion (impacto del bloqueo/overhead):"
echo "--- Tiempo Sin Mutex ---"
time ./carrera
echo -e "\n--- Tiempo Con Mutex ---"
time ./seguro
```

Desglose línea por línea

- **`if ! command -v gcc >/dev/null 2>&1; then ... exit 1; fi`**: verifica que el compilador de C (`gcc`) esté instalado; si no, indica cómo instalarlo y termina.
- **`gcc carrera.c -o carrera -lpthread`** y **`gcc seguro.c -o seguro -lpthread`**: compilan los dos programas en C. `-o` define el nombre del ejecutable y `-lpthread` agrega la librería de hilos de Linux, necesaria para crear hilos y usar Mutex.
- **`if [ $? -ne 0 ]; then ... exit 1; fi`**: `$?` guarda el resultado del último comando; si es distinto de `0` (`-ne 0`), la compilación falló y el script se detiene.
- **`./carrera`**: ejecuta la versión sin protección; el saldo obtenido es menor a 2.000.000.
- **`./seguro`**: ejecuta la versión con Mutex; el saldo obtenido es exactamente 2.000.000.
- **`time ./carrera`** y **`time ./seguro`**: vuelven a ejecutar ambos midiendo el tiempo. `real` es el tiempo total, `user` el tiempo en Modo Usuario y `sys` el tiempo en Modo Kernel (donde se nota el costo del Mutex).

En resumen

El script compila los dos programas, ejecuta primero el que tiene la condición de carrera y después el que está protegido con Mutex para comparar los saldos, y finalmente mide el tiempo de ambos para mostrar que la versión segura es correcta pero más lenta.

Código carrera.c (el problema en memoria)

unidad_2/carrera.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_HILOS 8
#define ITERACIONES 250000

long contador_compartido = 0;

void* depositar(void* arg) {
    for (int i = 0; i < ITERACIONES; i++) {
        // Sección crítica desprotegida (genera race condition)
        contador_compartido++;
    }
    return NULL;
}

int main() {
    pthread_t hilos[NUM_HILOS];

    for (int i = 0; i < NUM_HILOS; i++) {
        pthread_create(&hilos[i], NULL, depositar, NULL);
    }

    for (int i = 0; i < NUM_HILOS; i++) {
        pthread_join(hilos[i], NULL);
    }

    printf("[RACE CONDITION] Total esperado: %ld | Total obtenido: %ld\n",
           (long)NUM_HILOS * ITERACIONES, contador_compartido);
```

```
31 |
32 |     return 0;
33 | }
```

Desglose línea por línea

- **`#include <pthread.h>`**: incluye la librería de hilos.
- **`#define NUM_HILOS 8`** y **`#define ITERACIONES 250000`**: se crean 8 hilos y cada uno hace 250.000 depósitos (8 × 250.000 = 2.000.000).
- **`long contador_compartido = 0;`**: variable global que representa el saldo; todos los hilos usan la misma.
- **`void* depositar(void* arg)`**: función que ejecuta cada hilo. Repite 250.000 veces `contador_compartido++` sin ninguna protección, por lo que los hilos se pisan entre sí.
- **`pthread_create(&hilos[i], NULL, depositar, NULL);`**: crea los 8 hilos, todos ejecutando `depositar`.
- **`pthread_join(hilos[i], NULL);`**: el programa principal espera a que terminen los 8 hilos.
- **`printf("[RACE CONDITION] Total esperado: ... | Total obtenido: ...")`**: muestra el saldo esperado y el obtenido.

Código seguro.c (la versión corregida con Mutex)

unidad_2/seguro.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_HILOS 8
#define ITERACIONES 250000

long contador_compartido = 0;
pthread_mutex_t cerrojo;

void* depositar_seguro(void* arg) {
    for (int i = 0; i < ITERACIONES; i++) {
        // Sección de entrada
        pthread_mutex_lock(&cerrojo);

        // Sección crítica protegida
        contador_compartido++;

        // Sección de salida
        pthread_mutex_unlock(&cerrojo);
    }
    return NULL;
}

int main() {
    pthread_t hilos[NUM_HILOS];
    pthread_mutex_init(&cerrojo, NULL);

```

```
    for (int i = 0; i < NUM_HILOS; i++) {
        pthread_create(&hilos[i], NULL, depositar_seguro, NULL);
    }

    for (int i = 0; i < NUM_HILOS; i++) {
        pthread_join(hilos[i], NULL);
    }

    pthread_mutex_destroy(&cerrojo);

    printf("[MUTEX LOCK]     Total esperado: %ld | Total obtenido: %ld\n",
           (long)NUM_HILOS * ITERACIONES, contador_compartido);

    return 0;
}
```

Desglose línea por línea

- **`pthread_mutex_t cerrojo;`**: declara el Mutex que protege el saldo.
- **`pthread_mutex_init(&cerrojo, NULL);`**: inicializa el Mutex antes de crear los hilos.
- **`pthread_mutex_lock(&cerrojo);`** (sección de entrada): el hilo pide el cerrojo; si otro hilo lo tiene, espera.
- **`contador_compartido++;`** (sección crítica): con el cerrojo tomado, solo este hilo modifica el saldo.
- **`pthread_mutex_unlock(&cerrojo);`** (sección de salida): libera el cerrojo para el siguiente hilo.
- **`pthread_mutex_destroy(&cerrojo);`**: libera el Mutex al final, cuando ya terminaron todos los hilos.

Salida en consola al ejecutar ./demostracion_tpo.sh:

```
Seleccione el numero de unidad a ejecutar [1-6]: 2

=== DEMOSTRACION CONDICION DE CARRERA - UNIDAD 2 ===
Compilando programas en C...

1. Ejecutando programa con condicion de carrera (sin sincronizacion):
[RACE CONDITION] Total esperado: 2000000 | Total obtenido: 657038

2. Ejecutando programa seguro con pthread_mutex (con exclusion mutua):
[MUTEX LOCK]     Total esperado: 2000000 | Total obtenido: 2000000

3. Medicion de tiempos de ejecucion (impacto del bloqueo/overhead):
--- Tiempo Sin Mutex ---
[RACE CONDITION] Total esperado: 2000000 | Total obtenido: 538324

real    0m0.009s
user    0m0.024s
sys     0m0.003s

--- Tiempo Con Mutex ---
[MUTEX LOCK]     Total esperado: 2000000 | Total obtenido: 2000000

real    0m0.188s
user    0m0.219s
sys     0m0.470s
```

En resumen

Dos procesos escriben al mismo tiempo en el mismo archivo sin coordinarse y usando `>`, que reemplaza el contenido. El resultado es que el último en escribir borra el dato del anterior sin que el sistema muestre ningún error.

Cierre de la Unidad 2

Demostramos que cuando varios hilos o procesos modifican un mismo recurso sin coordinación se produce una condición de carrera: el saldo queda por debajo de 2.000.000 y, en el archivo, un proceso borra el dato del otro, todo sin que el sistema informe errores. La solución es la exclusión mutua: con un Mutex (`pthread_mutex_lock` / `pthread_mutex_unlock`) el saldo da exactamente 2.000.000 en todas las ejecuciones, y con `flock` y `>>` ambos procesos conservan su dato en el archivo. También comprobamos con `time` que esta seguridad tiene un costo: la versión con Mutex consume más tiempo en Modo Kernel (`sys`), porque el sistema operativo debe dormir y despertar a los hilos que esperan el cerrojo.

Unidad 3 — Asignación de Memoria Contigua y Fragmentación Externa

Introducción breve

En **Logistics Analytics Corp**, un servidor Red Hat ejecuta una base de datos que necesita reservar bloques **contiguos** de memoria RAM. La memoria libre está dividida en 5 huecos (100, 500, 200, 300 y 600 KB = 1.700 KB). Llegan 4 procesos que piden 212, 417, 112 y 426 KB. El sistema rechaza al proceso P4 (426 KB) por "memoria insuficiente", aunque después de ubicar a los otros tres quedan **959 KB libres**. Infraestructura culpaba a desarrollo por fugas de memoria y desarrollo decía que el sistema informaba mal el espacio libre.

Temas expuestos

- **Asignación contigua:** cada proceso debe ocupar un único bloque continuo de memoria.
- **Fragmentación externa:** hay memoria libre suficiente en total, pero está repartida en huecos chicos separados y ninguno alcanza por sí solo.
- **First-Fit (primer ajuste):** asigna el primer hueco donde entra el proceso. Es rápido, pero puede "romper" huecos grandes que después hacen falta.
- **Best-Fit (mejor ajuste):** revisa todos los huecos y elige aquel donde el proceso entra más justo, dejando libres los huecos grandes.
- **Paginación:** la solución que usa Linux en la realidad: divide la memoria en páginas de tamaño fijo (4096 bytes) para que un proceso no necesite un bloque contiguo.

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./ejecutar_tpo.sh`	Compila `simulador_memoria.c`, muestra el problema con First-Fit (P4 queda `RECHAZADO` con 959 KB libres), la solución con Best-Fit (entran los 4 procesos) y compara los tiempos de ambos algoritmos.

Código ejecutar_tpo.sh (Unidad 3)

unidad_3/ejecutar_tpo.sh

```
#!/bin/bash
# ============================================================================
# UNIDAD 3 - EJECUCION Y COMPARACION DE MEMORIA CONTIGUA (ejecutar_tpo.sh)
# Compila 'simulador_memoria.c' y compara dos algoritmos sobre los mismos 5 huecos
# de RAM (100, 500, 200, 300 y 600 KB) y 4 procesos (212, 417, 112 y 426 KB):
#   1) Modo F (First-Fit / Primer Ajuste): P4 (426 KB) es RECHAZADO por Fragmentacion Externa.
#   2) Modo B (Best-Fit / Mejor Ajuste)  : los 4 procesos logran entrar sin rechazos.
# ============================================================================

# 0. Verificamos que el compilador 'gcc' este instalado en el servidor Red Hat
if ! command -v gcc >/dev/null 2>&1; then
    echo "============================================================"
    echo " [ERROR]: 'gcc' no esta instalado en este sistema Red Hat."
    echo " Para instalarlo automaticamente junto a las demas herramientas:"
    echo "   cd .. && ./preparar_todo.sh"
    echo " O manualmente: sudo dnf install -y --disablerepo='*ansible*' gcc"
    echo "============================================================"
    exit 1
fi

echo "============================================================"
echo " COMPILANDO EL SIMULADOR EN C CON GCC (RED HAT)"
echo "============================================================"
# 1. Compilamos el simulador y verificamos con $? que no haya habido errores
gcc simulador_memoria.c -o simulador_memoria
if [ $? -ne 0 ]; then
    echo "[ERROR] Fallo critico en la compilacion."
    exit 1
fi
echo "[OK] Binario 'simulador_memoria' generado con exito."
echo ""

echo "============================================================"
echo " 1. EJECUCION CON PRIMER AJUSTE (FIRST-FIT)"
echo "============================================================"
# 2. Pasamos el parametro 'F': asigna el PRIMER hueco donde entra el proceso.
#    Aqui P4 (426 KB) falla aunque sobran 959 KB libres en total (Fragmentacion Externa).
./simulador_memoria F
echo ""

echo "============================================================"
echo " 2. EJECUCION CON MEJOR AJUSTE (BEST-FIT)"
echo "============================================================"
# 3. Pasamos el parametro 'B': revisa todos los huecos y elige el mas ajustado.
#    Preserva el hueco grande de 600 KB y permite que P4 entre con exito.
./simulador_memoria B
echo ""

```

```
echo "============================================================"
echo " 3. ANALISIS COMPARATIVO DE TIEMPOS DE EJECUCION"
echo "============================================================"
# 4. Medimos el tiempo con 'time'. Redirigimos la salida de texto a /dev/null
#    para ver limpia unicamente la medicion de tiempos (real, user y sys).
echo "--- Tiempo First-Fit ---"
time ./simulador_memoria F > /dev/null
echo ""
echo "--- Tiempo Best-Fit ---"
time ./simulador_memoria B > /dev/null
```

Desglose línea por línea

- **`if ! command -v gcc ...; fi`**: verifica que `gcc` esté instalado.
- **`gcc simulador_memoria.c -o simulador_memoria`**: compila el simulador en C.
- **`if [ $? -ne 0 ]; then ... exit 1; fi`**: si la compilación falló, detiene el script.
- **`./simulador_memoria F`**: ejecuta el simulador con la letra `F`, que le indica usar First-Fit (reproduce el problema).
- **`./simulador_memoria B`**: ejecuta el simulador con la letra `B`, que le indica usar Best-Fit (muestra la solución).
- **`time ./simulador_memoria F > /dev/null`** y **`time ./simulador_memoria B > /dev/null`**: miden el tiempo de cada algoritmo. `> /dev/null` descarta el texto del simulador para que solo se vea la medición.

En resumen

El script compila el simulador, lo ejecuta primero con First-Fit (donde aparece el rechazo de P4) y después con Best-Fit sobre los mismos datos (donde entran los 4 procesos), y finalmente compara el tiempo que tarda cada algoritmo.

Código simulador_memoria.c

unidad_3/simulador_memoria.c

```c
#include <stdio.h>
#include <string.h>

#define CANT_HUECOS 5
#define CANT_PROCESOS 4

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Uso: %s [F|B] (F: First-Fit, B: Best-Fit)\n", argv[0]);
        return 1;
    }

    char modo = argv[1][0];
    if (modo != 'F' && modo != 'f' && modo != 'B' && modo != 'b') {
        printf("Algoritmo no valido. Use F o B.\n");
        return 1;
    }

    // Tamaños iniciales de los huecos de memoria libre (en KB)
    int huecos[CANT_HUECOS] = {100, 500, 200, 300, 600};
    // Tamaños requeridos por los procesos entrantes (en KB)
    int procesos[CANT_PROCESOS] = {212, 417, 112, 426};
    // Arreglo para guardar qué hueco se le asignó a cada proceso (-1 = no asignado)
    int asignacion[CANT_PROCESOS];

    for (int i = 0; i < CANT_PROCESOS; i++) {
        asignacion[i] = -1;
    }

    printf("============================================================\n");
    if (modo == 'F' || modo == 'f') {
        printf(" SIMULACION: PRIMER AJUSTE (FIRST-FIT)\n");
        printf(" Logica: Se asigna el PRIMER hueco con capacidad suficiente.\n");
    } else {
        printf(" SIMULACION: MEJOR AJUSTE (BEST-FIT)\n");
        printf(" Logica: Se busca el hueco mas ajustado para reducir el sobrante.\n");
    }
    printf("============================================================\n");

    // 1. Mostrar estado inicial de los huecos y procesos antes de empezar
    printf("\n[ESTADO INICIAL DE LA MEMORIA LIBRE]\n");
    int memoria_inicial = 0;
    for (int j = 0; j < CANT_HUECOS; j++) {
        printf("Hueco %d: %d KB disponibles\n", j + 1, huecos[j]);
        memoria_inicial += huecos[j];
    }
    printf("Capacidad total inicial disponible: %d KB\n", memoria_inicial);

    printf("\n[PROCESOS QUE SOLICITAN MEMORIA CONTIGUA]\n");
    int memoria_solicitada = 0;
    for (int i = 0; i < CANT_PROCESOS; i++) {
```

```
        printf("Proceso P%d: %d KB requeridos\n", i + 1, procesos[i]);
        memoria_solicitada += procesos[i];
    }
    printf("Memoria total requerida: %d KB\n", memoria_solicitada);
    printf("------------------------------------------------------------\n");

    // 2. Ejecución del algoritmo seleccionado
    if (modo == 'F' || modo == 'f') {
        for (int i = 0; i < CANT_PROCESOS; i++) {
            for (int j = 0; j < CANT_HUECOS; j++) {
                if (huecos[j] >= procesos[i]) {
                    asignacion[i] = j;
                    huecos[j] -= procesos[i]; // El hueco se reduce
                    break; // Al encontrar el primero, corta la búsqueda
                }
            }
        }
    } else {
        for (int i = 0; i < CANT_PROCESOS; i++) {
            int mejor_idx = -1;
            for (int j = 0; j < CANT_HUECOS; j++) {
                if (huecos[j] >= procesos[i]) {
                    if (mejor_idx == -1 || huecos[j] < huecos[mejor_idx]) {
                        mejor_idx = j;
                    }
                }
            }
            if (mejor_idx != -1) {
                asignacion[i] = mejor_idx;
                huecos[mejor_idx] -= procesos[i];
            }
        }
    }

    // 3. Reporte de resultados de asignación
    printf("\n[RESULTADO DE LA ASIGNACION]\n");
    printf("Proceso\tTamano\tHueco Asignado\tEstado\n");
    for (int i = 0; i < CANT_PROCESOS; i++) {
        printf("P%d\t%d KB\t", i + 1, procesos[i]);
        if (asignacion[i] != -1) {
            printf("Hueco %d\t\tASIGNADO\n", asignacion[i] + 1);
        } else {
            printf("Ninguno\t\tRECHAZADO (Sin espacio contiguo)\n");
        }
    }

    // 4. Estado final de los huecos libres y fragmentación externa
    printf("\n[ESTADO FINAL DE LOS HUECOS LIBRES]\n");
     int memoria_desperdiciada = 0;
     for (int j = 0; j < CANT_HUECOS; j++) {
         printf("Hueco %d: %d KB disponibles\n", j + 1, huecos[j]);
         memoria_desperdiciada += huecos[j];
     }
```

```
105 |     printf("Memoria total libre no contigua remanente: %d KB\n", memoria_desperdiciada);
106 |
107 |     return 0;
108 | }
```

Desglose bloque por bloque

- **Validación del parámetro (`argc`, `argv[1][0]`):** el programa recibe la letra del algoritmo. Si no se pasa ninguna o no es `F` ni `B`, muestra cómo usarlo y termina con error.
- **Datos iniciales:** `huecos[5] = {100, 500, 200, 300, 600}` son los huecos libres en KB; `procesos[4] = {212, 417, 112, 426}` son los KB que pide cada proceso; `asignacion[4]` empieza en `-1` y guarda en qué hueco quedó cada proceso (`-1` = rechazado).
- **Estado inicial:** recorre los arreglos y muestra la memoria libre total (1.700 KB) y la memoria pedida (1.167 KB).
- **First-Fit:** para cada proceso recorre los huecos en orden; en el primero donde entra (`huecos[j] >= procesos[i]`) lo asigna, descuenta ese espacio y corta la búsqueda con `break`.
- **Best-Fit:** para cada proceso recorre **todos** los huecos y se queda con el más chico donde entra (`mejor_idx`). Recién al terminar de revisar todos, lo asigna y descuenta el espacio.
- **Resultados:** muestra una tabla con el hueco asignado a cada proceso (o `RECHAZADO (Sin espacio contiguo)`) y suma la memoria libre que quedó.

Salida en consola al ejecutar el ejercicio:

```
------------------------------------------------------------------
Seleccione el numero de unidad a ejecutar [1-6]: 3

============================================================
 COMPILANDO EL SIMULADOR EN C CON GCC (RED HAT)
============================================================
[OK] Binario 'simulador_memoria' generado con exito.

============================================================
 1. EJECUCION CON PRIMER AJUSTE (FIRST-FIT)
============================================================
============================================================
 SIMULACION: PRIMER AJUSTE (FIRST-FIT)
 Logica: Se asigna el PRIMER hueco con capacidad suficiente.
============================================================

[ESTADO INICIAL DE LA MEMORIA LIBRE]
Hueco 1: 100 KB disponibles
Hueco 2: 500 KB disponibles
Hueco 3: 200 KB disponibles
Hueco 4: 300 KB disponibles
Hueco 5: 600 KB disponibles
Capacidad total inicial disponible: 1700 KB

[PROCESOS QUE SOLICITAN MEMORIA CONTIGUA]
Proceso P1: 212 KB requeridos
Proceso P2: 417 KB requeridos
Proceso P3: 112 KB requeridos
Proceso P4: 426 KB requeridos
Memoria total requerida: 1167 KB
------------------------------------------------------------

[RESULTADO DE LA ASIGNACION]
Proceso Tamano  Hueco Asignado  Estado
P1      212 KB  Hueco 2         ASIGNADO
P2      417 KB  Hueco 5         ASIGNADO
P3      112 KB  Hueco 2         ASIGNADO
P4      426 KB  Ninguno         RECHAZADO (Sin espacio contiguo)

[ESTADO FINAL DE LOS HUECOS LIBRES]
Hueco 1: 100 KB disponibles
Hueco 2: 176 KB disponibles
Hueco 3: 200 KB disponibles
Hueco 4: 300 KB disponibles
Hueco 5: 183 KB disponibles
Memoria total libre no contigua remanente: 959 KB
```

```
============================================================
 2. EJECUCION CON MEJOR AJUSTE (BEST-FIT)
============================================================
============================================================
 SIMULACION: MEJOR AJUSTE (BEST-FIT)
 Logica: Se busca el hueco mas ajustado para reducir el sobrante.
============================================================

[ESTADO INICIAL DE LA MEMORIA LIBRE]
Hueco 1: 100 KB disponibles
Hueco 2: 500 KB disponibles
Hueco 3: 200 KB disponibles
Hueco 4: 300 KB disponibles
Hueco 5: 600 KB disponibles
Capacidad total inicial disponible: 1700 KB

[PROCESOS QUE SOLICITAN MEMORIA CONTIGUA]
Proceso P1: 212 KB requeridos
Proceso P2: 417 KB requeridos
Proceso P3: 112 KB requeridos
Proceso P4: 426 KB requeridos
Memoria total requerida: 1167 KB
------------------------------------------------------------

[RESULTADO DE LA ASIGNACION]
Proceso Tamano  Hueco Asignado  Estado
P1      212 KB  Hueco 4         ASIGNADO
P2      417 KB  Hueco 2         ASIGNADO
P3      112 KB  Hueco 3         ASIGNADO
P4      426 KB  Hueco 5         ASIGNADO

[ESTADO FINAL DE LOS HUECOS LIBRES]
Hueco 1: 100 KB disponibles
Hueco 2: 83 KB disponibles
Hueco 3: 88 KB disponibles
Hueco 4: 88 KB disponibles
Hueco 5: 174 KB disponibles
Memoria total libre no contigua remanente: 533 KB

============================================================
 3. ANALISIS COMPARATIVO DE TIEMPOS DE EJECUCION
============================================================
--- Tiempo First-Fit ---

real    0m0.001s
user    0m0.001s
sys     0m0.000s

--- Tiempo Best-Fit ---

real    0m0.001s
user    0m0.001s
sys     0m0.000s
```

* El tiempo que tarda cada uno de los metodos no se demuestra claramente debido al tamaño pequeño de los archivos que se usan en este ejercicio, en la practica real, First-Fit deberia ser mucho mas rapido que Best-Fit

Cierre de la Unidad 3

Con First-Fit, P1 (212 KB) ocupa el Hueco 2 de 500 KB, P2 (417 KB) ocupa el Hueco 5 de 600 KB y P3 (112 KB) vuelve a usar el Hueco 2. Cuando llega P4 (426 KB) ningún hueco individual alcanza y es rechazado, aunque quedan 959 KB libres en total: esto es la fragmentación externa. Con Best-Fit, cada proceso usa el hueco más justo (P1 en el Hueco 4, P2 en el 2, P3 en el 3) y se conserva el Hueco 5 de 600 KB, donde entra P4; los cuatro procesos quedan asignados. De todas formas, Best-Fit solo mejora el problema: los sistemas actuales como Red Hat lo eliminan con paginación, porque un proceso puede repartirse en páginas de 4096 bytes que no necesitan estar juntas.

Unidad 4 — Sistema de Archivos: Asignación Contigua vs. Inodos

Introducción breve

En **CloudVault Data Solutions**, un disco fue configurado con **asignación contigua**: cada archivo debe ocupar bloques seguidos. El disco tiene 5 huecos libres (100, 500, 200, 300 y 600 bloques). Al guardar 4 archivos de respaldo (212, 417, 112 y 426 bloques), el cuarto (`backup_transaccional.iso`) falla con el error `No space left on device (ENOSPC)`, aunque quedan **959 bloques libres**. Además, un archivo de registro existente no podía crecer porque el bloque siguiente estaba ocupado por otro archivo.

Temas expuestos

- **Asignación contigua en disco:** el archivo debe ocupar bloques consecutivos; sufre fragmentación externa y no permite que los archivos crezcan con facilidad.
- **Inodos (asignación indexada):** en Linux (`ext4`/`XFS`) cada archivo tiene un inodo, una ficha con sus datos (permisos, dueño, tamaño, fechas) y la lista de bloques donde está guardado. Los bloques pueden estar en cualquier parte del disco.
- **`ls -li` y `stat`:** muestran el número de inodo de un archivo y su información.
- **Número mágico (`file`):** Linux identifica el tipo de archivo leyendo sus primeros bytes, no por su extensión.

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./ejecutar_tpo.sh`	Compila `simulador_filesystem.c`, muestra el problema con asignación contigua First-Fit (el Archivo 4 da `NO ENTRA`), prueba Best-Fit, muestra la solución real con Inodos (entran todos repartidos en distintos huecos) e inspecciona un inodo real con `ls -li`, `stat` y `file`.

Código ejecutar_tpo.sh (Unidad 4)

unidad_4/ejecutar_tpo.sh

```
#!/bin/bash
# ================================================================================
# UNIDAD 4 - DEMOSTRACION DE SISTEMA DE ARCHIVOS E INODOS (ejecutar_tpo.sh)
# Compila 'simulador_filesystem.c' y compara como se guardan 4 archivos en disco:
#   - Modo F (Contiguo First-Fit): el Archivo 4 (426 bloques) falla con ENOSPC.
#   - Modo B (Contiguo Best-Fit) : entran al elegir el hueco mas justo.
#   - Modo I (Inodos Linux ext4) : reparte los bloques en cualquier hueco libre.
# Ademas inspecciona un inodo real con 'ls -li', 'stat' y numeros magicos con 'file'.
# ================================================================================

# 0. Verificamos que el compilador 'gcc' este instalado
if ! command -v gcc >/dev/null 2>&1; then
    echo "============================================================"
    echo " [ERROR]: 'gcc' no esta instalado en este sistema Red Hat."
    echo " Para instalarlo automaticamente junto a las demas herramientas:"
    echo "   cd .. && ./preparar_todo.sh"
    echo " O manualmente: sudo dnf install -y --disablerepo='*ansible*' gcc"
    echo "============================================================"
    exit 1
fi

echo "--- 1. COMPILANDO EL SIMULADOR DE DISCO ---"
gcc simulador_filesystem.c -o simulador_filesystem
if [ $? -ne 0 ]; then
    echo "[ERROR]: Fallo la compilacion."
    exit 1
fi
echo "[OK]: Compilado correctamente."
echo ""

echo "--- 2. PRUEBA CON ASIGNACION CONTIGUA (FIRST-FIT) ---"
# Parametro F: exige que todo el archivo entre de corrido en el primer hueco libre
./simulador_filesystem F
echo ""

echo "--- 3. PRUEBA CON ASIGNACION CONTIGUA (BEST-FIT) ---"
# Parametro B: exige bloques contiguos pero busca el hueco mas ajustado
./simulador_filesystem B
echo ""

echo "--- 4. SOLUCION LINUX: ASIGNACION CON INODOS (EXT4) ---"
# Parametro I: simula Inodos de Linux; fragmenta el archivo en bloques y usa todos los huecos
./simulador_filesystem I
echo ""

echo "--- 5. VIENDO UN INODO REAL EN LINUX (ls -li y stat) ---"
# 'ls -li': la opcion '-i' muestra en la primera columna el numero unico de Inodo del archivo
ls -li simulador_filesystem.c
echo ""
# 'stat': muestra todos los metadatos guardados dentro del Inodo (tamano, bloques, permisos, fechas)
stat simulador_filesystem.c
```

```
52 | echo ""
53 |
54 | echo "--- 6. VIENDO EL TIPO DE ARCHIVO POR NUMERO MAGICO (file) ---"
55 | # 'file' lee los primeros bytes (Numero Magico) para distinguir texto ASCII de un binario ELF
56 | file simulador_filesystem.c
57 | file simulador_filesystem
```

Desglose línea por línea

- **`if ! command -v gcc ...; fi`**: verifica que `gcc` esté instalado.
- **`gcc simulador_filesystem.c -o simulador_filesystem`**: compila el simulador del disco.
- **`./simulador_filesystem F`**: asignación contigua con First-Fit; el Archivo 4 no entra (el problema).
- **`./simulador_filesystem B`**: asignación contigua con Best-Fit; los archivos entran porque cada uno usa el hueco más justo.
- **`./simulador_filesystem I`**: asignación con inodos (la solución de Linux); cada archivo se reparte en varios huecos libres.
- **`ls -li simulador_filesystem.c`**: `-i` muestra en la primera columna el número de inodo real del archivo.
- **`stat simulador_filesystem.c`**: muestra la información del inodo: número de inodo, tamaño, bloques ocupados, tamaño de bloque (`IO Block: 4096`), permisos, dueño y fechas.
- **`file simulador_filesystem.c`** y **`file simulador_filesystem`**: identifican el tipo real de cada archivo: el primero es código fuente en texto y el segundo un ejecutable de Linux (`ELF 64-bit`).

En resumen

El script compila el simulador y lo ejecuta en los tres modos para comparar el problema de la asignación contigua frente a la solución por inodos. Luego usa `ls -li`, `stat` y `file` para mostrar cómo Linux guarda la información de un archivo real en su inodo y cómo reconoce su tipo.

Código simulador_filesystem.c

unidad_4/simulador_filesystem.c

```
 1 | #include <stdio.h>
 2 |
 3 | #define CANT_HUECOS 5
 4 | #define CANT_ARCHIVOS 4
 5 |
 6 | int main(int argc, char *argv[]) {
 7 |     if (argc < 2) {
 8 |         printf("Uso: %s [F|B|I] (F: First-Fit Contiguo, B: Best-Fit Contiguo, I: Inodos Linux)\n",
argv[0]);
 9 |         return 1;
10 |     }
11 |
12 |     char modo = argv[1][0];
13 |     // Bloques libres en el disco (5 huecos separados, total = 1700 bloques)
14 |     int huecos[CANT_HUECOS] = {100, 500, 200, 300, 600};
15 |     // Tamano de los 4 archivos que queremos guardar en el disco (en bloques)
16 |     int archivos[CANT_ARCHIVOS] = {212, 417, 112, 426};
17 |     int asignacion[CANT_ARCHIVOS] = {-1, -1, -1, -1};
18 |
19 |     printf("============================================================\n");
```

```c
    if (modo == 'F' || modo == 'f') {
        printf(" MODO [F]: ASIGNACION CONTIGUA - PRIMER AJUSTE (FIRST-FIT)\n");
        printf(" Regla: Todo el archivo debe entrar junto en un solo hueco.\n");
    } else if (modo == 'B' || modo == 'b') {
        printf(" MODO [B]: ASIGNACION CONTIGUA - MEJOR AJUSTE (BEST-FIT)\n");
        printf(" Regla: Todo el archivo entra junto en el hueco mas justo.\n");
    } else {
        printf(" MODO [I]: ASIGNACION CON INODOS (COMO USA LINUX EXT4)\n");
        printf(" Regla: El archivo se puede repartir en varios huecos libres.\n");
    }
    printf("============================================================\n\n");

    // 1. Ejecucion segun el modo elegido
    if (modo == 'F' || modo == 'f') {
        for (int i = 0; i < CANT_ARCHIVOS; i++) {
            for (int j = 0; j < CANT_HUECOS; j++) {
                if (huecos[j] >= archivos[i]) {
                    asignacion[i] = j;
                    huecos[j] -= archivos[i];
                    break; // Corta en el primer hueco donde entra
                }
            }
        }
    } else if (modo == 'B' || modo == 'b') {
        for (int i = 0; i < CANT_ARCHIVOS; i++) {
            int mejor = -1;
            for (int j = 0; j < CANT_HUECOS; j++) {
                if (huecos[j] >= archivos[i]) {
                    if (mejor == -1 || huecos[j] < huecos[mejor]) {
                        mejor = j;
                    }
                }
            }
            if (mejor != -1) {
                asignacion[i] = mejor;
                huecos[mejor] -= archivos[i];
            }
        }
    } else if (modo == 'I' || modo == 'i') {
        // Modo Inodos: cada archivo toma bloques de cualquier hueco libre disponible
        for (int i = 0; i < CANT_ARCHIVOS; i++) {
            int falta_guardar = archivos[i];
            for (int j = 0; j < CANT_HUECOS && falta_guardar > 0; j++) {
                if (huecos[j] > 0) {
                    int usar = (huecos[j] >= falta_guardar) ? falta_guardar : huecos[j];
                    huecos[j] -= usar;
                    falta_guardar -= usar;
                }
            }
            if (falta_guardar == 0) {
                asignacion[i] = 99; // 99 = Guardado con exito usando un Inodo
            }
        }
```

```
    }

    // 2. Mostrar resultados
    printf("[RESULTADO AL GUARDAR LOS ARCHIVOS EN EL DISCO]\n");
    printf("Archivo\t\tTamano\t\tEstado en el Disco\n");
    for (int i = 0; i < CANT_ARCHIVOS; i++) {
        printf("Archivo %d\t%d bloques\t", i + 1, archivos[i]);
        if (asignacion[i] == 99) {
            printf("GUARDADO (Inodo repartio los bloques en huecos libres)\n");
        } else if (asignacion[i] != -1) {
            printf("GUARDADO en Hueco %d (Contiguo)\n", asignacion[i] + 1);
        } else {
            printf("ERROR: NO ENTRA (Falta espacio contiguo!)\n");
        }
    }

    // 3. Mostrar cuanto espacio libre quedo en el disco
    int total_libre = 0;
    printf("\n[ESPACIO LIBRE QUE QUEDO EN CADA HUECO DEL DISCO]\n");
    for (int j = 0; j < CANT_HUECOS; j++) {
        printf("Hueco %d: %d bloques libres\n", j + 1, huecos[j]);
        total_libre += huecos[j];
    }
    printf("-> Total de espacio libre sobrante en el disco: %d bloques\n", total_libre);

    return 0;
}
```

Desglose bloque por bloque

- **Datos iniciales:** `huecos[5] = {100, 500, 200, 300, 600}` son los bloques libres del disco y `archivos[4] = {212, 417, 112, 426}` el tamaño de cada archivo en bloques.
- **Modos `F` y `B` (contiguos):** un archivo solo se guarda si entra completo en un único hueco (`huecos[j] >= archivos[i]`). `F` usa el primer hueco que sirve y `B` el más justo.
- **Modo `I` (inodos):** para cada archivo guarda en `falta_guardar` cuántos bloques necesita. Recorre los huecos y toma los bloques libres que haya en cada uno, aunque no alcancen para todo el archivo (`usar = (huecos[j] >= falta_guardar) ? falta_guardar : huecos[j]`). Sigue con el próximo hueco hasta completar el archivo y lo marca como guardado (`asignacion[i] = 99`).
- **Resultados:** muestra el estado de cada archivo y los bloques libres que quedaron.

Salida en consola al ejecutar el ejercicio:

```
--- 1. COMPILANDO EL SIMULADOR DE DISCO ---
[OK]: Compilado correctamente.

--- 2. PRUEBA CON ASIGNACION CONTIGUA (FIRST-FIT) ---
============================================================
 MODO [F]: ASIGNACION CONTIGUA - PRIMER AJUSTE (FIRST-FIT)
 Regla: Todo el archivo debe entrar junto en un solo hueco.
============================================================

[RESULTADO AL GUARDAR LOS ARCHIVOS EN EL DISCO]
Archivo         Tamano          Estado en el Disco
Archivo 1       212 bloques     GUARDADO en Hueco 2 (Contiguo)
Archivo 2       417 bloques     GUARDADO en Hueco 5 (Contiguo)
Archivo 3       112 bloques     GUARDADO en Hueco 2 (Contiguo)
Archivo 4       426 bloques     ERROR: NO ENTRA (Falta espacio contiguo!)

[ESPACIO LIBRE QUE QUEDO EN CADA HUECO DEL DISCO]
Hueco 1: 100 bloques libres
Hueco 2: 176 bloques libres
Hueco 3: 200 bloques libres
Hueco 4: 300 bloques libres
Hueco 5: 183 bloques libres
-> Total de espacio libre sobrante en el disco: 959 bloques

--- 3. PRUEBA CON ASIGNACION CONTIGUA (BEST-FIT) ---
============================================================
 MODO [B]: ASIGNACION CONTIGUA - MEJOR AJUSTE (BEST-FIT)
 Regla: Todo el archivo entra junto en el hueco mas justo.
============================================================

[RESULTADO AL GUARDAR LOS ARCHIVOS EN EL DISCO]
Archivo         Tamano          Estado en el Disco
Archivo 1       212 bloques     GUARDADO en Hueco 4 (Contiguo)
Archivo 2       417 bloques     GUARDADO en Hueco 2 (Contiguo)
Archivo 3       112 bloques     GUARDADO en Hueco 3 (Contiguo)
Archivo 4       426 bloques     GUARDADO en Hueco 5 (Contiguo)

[ESPACIO LIBRE QUE QUEDO EN CADA HUECO DEL DISCO]
Hueco 1: 100 bloques libres
Hueco 2: 83 bloques libres
Hueco 3: 88 bloques libres
Hueco 4: 88 bloques libres
Hueco 5: 174 bloques libres
-> Total de espacio libre sobrante en el disco: 533 bloques
```

```
[RESULTADO AL GUARDAR LOS ARCHIVOS EN EL DISCO]
Archivo         Tamano          Estado en el Disco
Archivo 1       212 bloques     GUARDADO en Hueco 4 (Contiguo)
Archivo 2       417 bloques     GUARDADO en Hueco 2 (Contiguo)
Archivo 3       112 bloques     GUARDADO en Hueco 3 (Contiguo)
Archivo 4       426 bloques     GUARDADO en Hueco 5 (Contiguo)

[ESPACIO LIBRE QUE QUEDO EN CADA HUECO DEL DISCO]
Hueco 1: 100 bloques libres
Hueco 2: 83 bloques libres
Hueco 3: 88 bloques libres
Hueco 4: 88 bloques libres
Hueco 5: 174 bloques libres
-> Total de espacio libre sobrante en el disco: 533 bloques

--- 4. SOLUCION LINUX: ASIGNACION CON INODOS (EXT4) ---
============================================================
 MODO [I]: ASIGNACION CON INODOS (COMO USA LINUX EXT4)
 Regla: El archivo se puede repartir en varios huecos libres.
============================================================

[RESULTADO AL GUARDAR LOS ARCHIVOS EN EL DISCO]
Archivo         Tamano          Estado en el Disco
Archivo 1       212 bloques     GUARDADO (Inodo repartio los bloques en huecos libres)
Archivo 2       417 bloques     GUARDADO (Inodo repartio los bloques en huecos libres)
Archivo 3       112 bloques     GUARDADO (Inodo repartio los bloques en huecos libres)
Archivo 4       426 bloques     GUARDADO (Inodo repartio los bloques en huecos libres)

[ESPACIO LIBRE QUE QUEDO EN CADA HUECO DEL DISCO]
Hueco 1: 0 bloques libres
Hueco 2: 0 bloques libres
Hueco 3: 0 bloques libres
Hueco 4: 0 bloques libres
Hueco 5: 533 bloques libres
-> Total de espacio libre sobrante en el disco: 533 bloques

--- 5. VIENDO UN INODO REAL EN LINUX (ls -li y stat) ---
34191358 -rw-r--r--. 1 student student 3980 Nov  1 03:21 simulador_filesystem.c

  File: simulador_filesystem.c
  Size: 3980            Blocks: 8          IO Block: 4096   regular file
Device: fc04h/64516d    Inode: 34191358    Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ student)   Gid: ( 1000/ student)
Context: unconfined_u:object_r:user_home_t:s0
Access: 2026-11-01 03:21:59.938208829 +0000
Modify: 2026-11-01 03:21:31.703068834 +0000
Change: 2026-11-01 03:21:31.703068834 +0000
 Birth: 2026-11-01 03:21:31.703068834 +0000

--- 6. VIENDO EL TIPO DE ARCHIVO POR NUMERO MAGICO (file) ---
simulador_filesystem.c: C source, ASCII text
simulador_filesystem: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=638f108433c529ebfbb2e29fca92d0449bf34c27, for GNU/Linux 3.2.0, not stripped
```

Cierre de la Unidad 4

Con asignación contigua, el Archivo 4 (426 bloques) no puede guardarse aunque quedan 959 bloques libres, porque ningún hueco individual alcanza: es la fragmentación externa en disco. Con inodos, el sistema de archivos reparte los bloques de un mismo archivo en distintos huecos libres y anota en el inodo dónde está cada parte, por lo que los cuatro archivos se guardan y pueden crecer sin necesidad de moverse en el disco. Con `ls -li`, `stat` y `file` también vimos cómo Linux guarda la información de cada archivo en su inodo y cómo reconoce su tipo por el número mágico.

Unidad 5 — Entrada/Salida y Uso de Búfer

Introducción breve

En **AeroData Telemetry Corp**, un servicio escrito en C guarda datos en disco en un servidor Red Hat. Tras ponerlo en producción, el servidor se volvió lento y el procesador pasaba la mayor parte del tiempo en Modo Kernel. La causa es que el programa escribe **de a 1 byte por vez**: para guardar 100.000 caracteres hace 100.000 llamadas al sistema `write()`. Se busca demostrar la diferencia con una versión que usa un **búfer** de 4.000 bytes.

Temas expuestos

- **Costo de cada llamada al sistema:** cada `write()` obliga al procesador a pasar de Modo Usuario a Modo Kernel y volver. Hacerlo miles de veces para mover muy pocos datos genera una gran pérdida de tiempo.
- **Búfer (*buffering*):** un espacio en memoria donde se juntan los datos para enviarlos al Kernel todos juntos, en pocas llamadas.
- **Medición con `time` y `strace -c`:** `time` mide el tiempo en Modo Kernel (`sys`) y `strace -c` cuenta cuántas llamadas al sistema se hicieron.

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./ejecutar_tpo.sh`	Compila ambos programas, mide el tiempo de `sin_bufer` (el problema) y `con_bufer` (la solución) y cuenta con `strace -c` las llamadas a `write` (100.001 frente a 26).

Código ejecutar_tpo.sh (Unidad 5)

unidad_5/ejecutar_tpo.sh

```
#!/bin/bash
# ============================================================================
# UNIDAD 5 - DEMOSTRACION DE ENTRADA/SALIDA Y USO DE BUFER (ejecutar_tpo.sh)
# Compila y compara dos programas en C que escriben el mismo archivo de 100.000 bytes:
#   1) sin_bufer.c : escribe de a 1 byte por vez (100.000 llamadas a write -> Lento)
#   2) con_bufer.c : junta bloques de 4.000 bytes en RAM (solo 25 llamadas a write -> Rapido)
# ============================================================================

# 0. Verificamos que tanto 'gcc' como 'strace' esten instalados en Red Hat
if ! command -v gcc >/dev/null 2>&1 || ! command -v strace >/dev/null 2>&1; then
    echo "============================================================"
    echo " [ERROR]: 'gcc' o 'strace' no estan instalados en Red Hat."
    echo " Para instalar todo automaticamente:"
    echo "   cd .. && ./preparar_todo.sh"
    echo " O manualmente: sudo dnf install -y --disablerepo='*ansible*' gcc strace"
    echo "============================================================"
    exit 1
```

```
fi

echo "--- 1. COMPILANDO LOS DOS PROGRAMAS EN C ---"
# Compilamos ambos programas encadenados con '&&' (el segundo compila si el primero salio bien)
gcc sin_bufer.c -o sin_bufer && gcc con_bufer.c -o con_bufer
if [ $? -ne 0 ]; then
    echo "[ERROR]: Fallo la compilacion."
    exit 1
fi
echo "[OK]: Compilados 'sin_bufer' y 'con_bufer'."
echo ""

echo "--- 2. EJECUCION LENTA: SIN BUFER (100.000 viajes al Kernel) ---"
# Medimos con 'time' cuanto tarda hacer 100.000 cambios de contexto (Modo Usuario <-> Modo Kernel)
time ./sin_bufer
echo ""

echo "--- 3. EJECUCION RAPIDA: CON BUFER (Solo 25 viajes al Kernel) ---"
# Medimos con 'time' cuanto tarda cuando juntamos de a 4.000 bytes en un bufer en memoria
time ./con_bufer
echo ""

echo "--- 4. CONTANDO LAS LLAMADAS AL SISTEMA 'write' CON STRACE ---"
# 'strace -c -e trace=write' cuenta exactamente cuantas veces cada programa invoco la syscall write
echo "-> Cantidad de syscalls SIN BUFER:"
strace -c -e trace=write ./sin_bufer > /dev/null
echo ""
echo "-> Cantidad de syscalls CON BUFER:"
strace -c -e trace=write ./con_bufer > /dev/null

# Borramos los archivos temporales de salida al terminar la demostracion
rm -f salida_lenta.txt salida_rapida.txt
```

Desglose línea por línea

- **`if ! command -v gcc ... || ! command -v strace ...; then ... fi`**: verifica que estén instalados `gcc` y `strace`. El `||` significa "o": si falta cualquiera de los dos, se detiene.
- **`gcc sin_bufer.c -o sin_bufer && gcc con_bufer.c -o con_bufer`**: compila los dos programas. El `&&` hace que el segundo solo se compile si el primero salió bien.
- **`time ./sin_bufer`** y **`time ./con_bufer`**: ejecutan ambos y miden su tiempo, en especial el `sys` (Modo Kernel).
- **`strace -c -e trace=write ./sin_bufer > /dev/null`** y lo mismo con `./con_bufer`: cuentan solo las llamadas a `write` de cada programa.
- **`rm -f salida_lenta.txt salida_rapida.txt`**: borra los archivos generados en la prueba.

En resumen

El script compila las dos versiones, mide cuánto tarda cada una y cuenta con `strace` cuántas veces cada programa llamó a `write`, mostrando en una sola ejecución el problema de escribir de a un byte y la solución de escribir con búfer.

Código sin_bufer.c (el problema)

unidad_5/sin_bufer.c

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

// MODO LENTO (SIN BUFER): Escribe 100.000 letras de a 1 por vez.
// Cada write() hace un cambio de contexto al Kernel (100.000 viajes).

int main() {
    int fd = open("salida_lenta.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) return 1; // Validamos que el archivo se abrio correctamente

    printf("[SIN BUFER]: Escribiendo 100.000 letras de a 1 byte por vez...\n");
    for (int i = 0; i < 100000; i++) {
        write(fd, "A", 1); // 1 llamada al sistema por cada letra
    }

    close(fd);
    printf("[SIN BUFER]: Terminado (100.000 llamadas a write).\n");
    return 0;
}
```

Desglose línea por línea

- **`#include <fcntl.h>`** y **`#include <unistd.h>`**: incluyen las funciones `open`, `write` y `close`, que llaman directamente al Kernel.
- **`int fd = open("salida_lenta.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);`**: abre el archivo para escribir (`O_WRONLY`), lo crea si no existe (`O_CREAT`) y lo vacía si ya tenía contenido (`O_TRUNC`). Guarda en `fd` el descriptor del archivo.
- **`if (fd < 0) return 1;`**: si no se pudo abrir, termina con error.
- **`for (...100000...) { write(fd, "A", 1); }`**: escribe la letra `A` de a 1 byte, 100.000 veces: son 100.000 llamadas al Kernel.
- **`close(fd);`**: cierra el archivo.

Código con_bufer.c (la solución)

unidad_5/con_bufer.c

```c
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

// MODO RAPIDO (CON BUFER): Junta 4000 letras en memoria y escribe en bloque.
// En vez de 100.000 viajes al Kernel, hace solo 25 viajes (25 x 4000 = 100.000).

int main() {
    int fd = open("salida_rapida.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) return 1; // Validamos que el archivo se abrio correctamente

    char bufer[4000];
    memset(bufer, 'A', 4000); // Llenamos el bufer con 4000 letras 'A'

    printf("[CON BUFER]: Escribiendo 100.000 letras en bloques de 4000 bytes...\n");
    for (int i = 0; i < 25; i++) {
        write(fd, bufer, 4000); // 1 sola llamada escribe 4000 letras juntas
    }

    close(fd);
    printf("[CON BUFER]: Terminado (Solo 25 llamadas a write!).\n");
    return 0;
}
```

Desglose línea por línea

- **`char bufer[4000];`** y **`memset(bufer, 'A', 4000);`**: crea un búfer de 4.000 bytes en memoria y lo llena con letras `A`, sin intervención del Kernel.
- **`for (int i = 0; i < 25; i++) { write(fd, bufer, 4000); }`**: envía el búfer completo 25 veces (25 × 4.000 = 100.000 bytes): solo 25 llamadas al Kernel.
- **`close(fd);`**: cierra el archivo.

Salida en consola al ejecutar el ejercicio:

```
----------------------------------------------------------------
Seleccione el numero de unidad a ejecutar [1-6]: 5

--- 1. COMPILANDO LOS DOS PROGRAMAS EN C ---
[OK]: Compilados 'sin_bufer' y 'con_bufer'.

--- 2. EJECUCION LENTA: SIN BUFER (100.000 viajes al Kernel) ---
[SIN BUFER]: Escribiendo 100.000 letras de a 1 byte por vez...
[SIN BUFER]: Terminado (100.000 llamadas a write).

real    0m0.047s
user    0m0.002s
sys     0m0.045s

--- 3. EJECUCION RAPIDA: CON BUFER (Solo 25 viajes al Kernel) ---
[CON BUFER]: Escribiendo 100.000 letras en bloques de 4000 bytes...
[CON BUFER]: Terminado (Solo 25 llamadas a write!).

real    0m0.001s
user    0m0.001s
sys     0m0.000s

--- 4. CONTANDO LAS LLAMADAS AL SISTEMA 'write' CON STRACE ---
-> Cantidad de syscalls SIN BUFER:
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.711775           7    100001           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.711775           7    100001           total

-> Cantidad de syscalls CON BUFER:
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.000344          13        26           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000344          13        26           total
```

Cierre de la Unidad 5

Las dos versiones escriben exactamente el mismo archivo de 100.000 bytes, pero la versión sin búfer hace 100.001 llamadas a `write` (100.000 por las letras y una por el mensaje final), mientras que la versión con búfer hace solo 26 (25 bloques y una por los mensajes). Esto representa un 99,97 % menos de pasajes entre Modo Usuario y Modo Kernel, y el tiempo `sys` baja prácticamente a cero. La conclusión es que en Entrada/Salida no conviene pedirle al Kernel muchas operaciones pequeñas: juntar los datos en un búfer y escribirlos en bloques reduce drásticamente el costo.

Unidad 6 — Seguridad: Firewall, Contraseñas, ACL e Integridad

Introducción breve

Una empresa financiera ejecuta todos los días en un servidor Red Hat el script `pagos_empresa.sh`, que realiza la transferencia de sueldos. Un intruso (o un virus) logró modificar el archivo agregándole una línea maliciosa, manteniendo el mismo nombre. Si el administrador lo ejecuta confiando solo en el nombre, el código malicioso se ejecuta. Además, la empresa necesita controlar qué puertos de red están abiertos, guardar las contraseñas de forma segura y dar acceso a un reporte confidencial solo a un usuario auditor.

Temas expuestos

- **Firewall (`firewall-cmd`):** permite abrir o bloquear puertos para controlar qué conexiones de red se aceptan.
- **Contraseñas con Hash y Salt (`/etc/shadow`):** Linux no guarda las contraseñas, sino un resumen irreversible (hash) combinado con un valor aleatorio (salt), para que no puedan descubrirse aunque se lea el archivo.
- **Listas de Control de Acceso (ACL):** con `setfacl` y `getfacl` se puede dar permiso a un usuario puntual sobre un archivo sin abrirlo al resto.
- **Integridad con Hash SHA-256 (`sha256sum`):** se calcula una "huella digital" del archivo; si se modifica aunque sea un carácter, la huella cambia completamente y se detecta la alteración.
- **Defensa en profundidad:** combinar varias capas de protección (red, usuarios, permisos e integridad).

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./ejecutar_tpo.sh`	Prueba conectividad y firewall entre `servera` (puerto 8080 bloqueado) y `serverb` (puerto 8080 abierto), crea el usuario `auditor` mostrando su contraseña como Hash + Salt en `/etc/shadow`, le otorga acceso de lectura exclusivo con ACL (`setfacl`/`getfacl`) y demuestra cómo el comprobador de integridad (`sha256sum -c`) detecta y bloquea la modificación maliciosa de `pagos_empresa.sh`.

Código pagos_empresa.sh (el script protegido)

unidad_6/pagos_empresa.sh

```
#!/bin/bash
# ================================================================================
# UNIDAD 6 - SCRIPT CRITICO DE LA EMPRESA (pagos_empresa.sh)
# Simula la transferencia de sueldos de la empresa. Es el archivo que protegeremos
# con permisos 750 y con un Comprobador de Integridad Criptografico (Hash SHA-256).
# ================================================================================
```

```
7 | echo "[SISTEMA DE PAGOS]: Conectando con el banco..."
8 | echo "[SISTEMA DE PAGOS]: Transferencia de sueldos realizada con exito."
```

Desglose línea por línea

- **`echo "[SISTEMA DE PAGOS]: Conectando con el banco..."`** y **`echo "[SISTEMA DE PAGOS]: Transferencia de sueldos realizada con exito."`**: simulan la conexión con el banco y la transferencia de sueldos. Es el archivo crítico de la empresa que el intruso modifica y que debemos proteger.

Código ejecutar_tpo.sh (Unidad 6)

unidad_6/ejecutar_tpo.sh

```
1 | #!/bin/bash
2 | # ================================================================================
3 | # UNIDAD 6 - DEMOSTRACION COMPLETA DE SEGURIDAD EN RED HAT (ejecutar_tpo.sh)
4 | # Reune en un unico script las 4 capas de seguridad:
5 | #   1) Red y Firewall entre servidores (ping, ssh y firewall-cmd en servera y serverb)
6 | #   2) Creacion del usuario 'auditor' y contrasena con Hash + Salt en /etc/shadow
7 | #   3) Control de Acceso con ACL (setfacl / getfacl) sobre un archivo confidencial
8 | #   4) Comprobador de Integridad con Hash SHA-256 (sha256sum -c) ante un virus
9 | # ================================================================================
10 |
11 | PUERTO=8080
12 |
13 | echo "================================================================================"
14 | echo " 1. CONECTIVIDAD Y FIREWALL ENTRE SERVIDORES ('servera' Y 'serverb')"
15 | echo "================================================================================"
16 | # 'ping -c 2' envia 2 paquetes para comprobar que ambos servidores responden en la red
17 | echo "-> Probando conexion de red con servera y serverb:"
18 | ping -c 2 servera
19 | echo ""
20 | ping -c 2 serverb
21 | echo ""
22 |
23 | # Nos conectamos por SSH a 'servera' y cerramos el puerto 8080/tcp en su Firewall
24 | echo "-> [servera]: Bloqueando el puerto $PUERTO/tcp en el Firewall..."
25 | ssh student@servera "echo student | sudo -S firewall-cmd --remove-port=${PUERTO}/tcp >/dev/null
2>&1; echo student | sudo -S firewall-cmd --list-ports 2>/dev/null"
26 |
27 | # Nos conectamos por SSH a 'serverb', abrimos el puerto 8080/tcp y levantamos un servidor web de
prueba
28 | echo "-> [serverb]: Abriendo el puerto $PUERTO/tcp en el Firewall y levantando servicio..."
29 | ssh student@serverb "echo student | sudo -S firewall-cmd --add-port=${PUERTO}/tcp >/dev/null 2>&1;
echo student | sudo -S firewall-cmd --list-ports 2>/dev/null; nohup python3 -m http.server $PUERTO
</dev/null >/dev/null 2>&1 &"
30 | sleep 2
31 | echo ""
32 |
33 | echo "--- PRUEBA DE CONEXION AL PUERTO $PUERTO ---"
34 | # 'curl' intenta conectarse por HTTP al puerto 8080 de ambos servidores:
35 | # En 'servera' falla porque su Firewall bloquea el puerto; en 'serverb' entra porque esta permitido.
36 | echo "1) Intentando conectar a servera:$PUERTO (Puerto BLOQUEADO por Firewall):"
```

```
if curl -s --connect-timeout 2 http://servera:$PUERTO >/dev/null; then
    echo "   [OK]: Conectado a servera."
else
    echo "   [BLOQUEADO]: El Firewall de 'servera' rechazo la conexion al puerto $PUERTO."
fi

echo "2) Intentando conectar a serverb:$PUERTO (Puerto HABILITADO en Firewall):"
if curl -s --connect-timeout 2 http://serverb:$PUERTO >/dev/null; then
    echo "   [PERMITIDO]: Conexion exitosa con 'serverb' en el puerto $PUERTO."
else
    echo "   [AVISO]: No se pudo conectar a serverb:$PUERTO."
fi
echo ""

echo "============================================================================="
echo " 2. CREACION DE USUARIO ('auditor') Y CONTRASENA CON HASH EN /etc/shadow"
echo "============================================================================="
# 'useradd -m' crea el usuario 'auditor' con su carpeta personal (/home/auditor)
sudo useradd -m auditor 2>/dev/null
# 'chpasswd' asigna la contrasena; Linux guarda su Hash + Salt en /etc/shadow
echo "auditor:RedHat2026" | sudo chpasswd
echo "[OK]: Usuario 'auditor' creado. Contrasena guardada como Hash + Salt en /etc/shadow:"
sudo grep "^auditor:" /etc/shadow
echo ""

echo "============================================================================="
echo " 3. LISTA DE CONTROL DE ACCESO (ACL) PARA EL USUARIO 'auditor'"
echo "============================================================================="
ARCHIVO="/tmp/reporte_confidencial.txt"
echo "DATOS SECRETOS DEL SERVIDOR RED HAT" > $ARCHIVO
# 'chmod 600' cierra el archivo: solo el dueno puede leer y escribir; el resto no tiene permisos
chmod 600 $ARCHIVO

echo "-> Otorgando permiso de lectura SOLO a 'auditor' mediante ACL (setfacl):"
# 'setfacl -m u:auditor:r--' agrega un permiso de lectura exclusivo para 'auditor'
setfacl -m u:auditor:r-- $ARCHIVO
# 'ls -l' muestra un signo '+' al final de los permisos indicando que tiene una ACL activa
ls -l $ARCHIVO
# 'getfacl' muestra el detalle de la ACL aplicada
getfacl $ARCHIVO
echo "-> Probando leer el archivo con el usuario 'auditor':"
sudo su - auditor -c "cat $ARCHIVO"
rm -f $ARCHIVO
echo ""

echo "============================================================================="
echo " 4. INTEGRIDAD CON HASH SHA-256 ANTE MODIFICACION DE 'pagos_empresa.sh'"
echo "============================================================================="
# Nos aseguramos de que pagos_empresa.sh este sano y con permisos 750
sed -i '/VIRUS/d' pagos_empresa.sh
chmod 750 pagos_empresa.sh

echo "-> 4.1 Guardamos la huella digital oficial (SHA-256) de pagos_empresa.sh sano:"
```

```
90 | # 'sha256sum' calcula el resumen de 256 bits del archivo original y lo guarda en 'sumacomp.txt'
91 | sha256sum pagos_empresa.sh > sumacomp.txt
92 | cat sumacomp.txt
93 | echo ""
94 |
95 | echo "-> 4.2 Ejecucion normal (verificando la huella antes de correr):"
96 | # 'sha256sum -c' recalcula la huella actual y la compara con sumacomp.txt; si da OK, ejecuta el
script
97 | if sha256sum -c sumacomp.txt; then
98 |     echo "[AUTORIZADO]: La huella coincide. Ejecutando pagos_empresa.sh:"
99 |     ./pagos_empresa.sh
100 | fi
101 | echo ""
102 |
103 | echo "-> 4.3 EL PROBLEMA: Un intruso/virus agrega una linea maliciosa a pagos_empresa.sh:"
104 | echo 'echo "[VIRUS]: Robando contrasenas..."' >> pagos_empresa.sh
105 | echo "Ultima linea agregada al archivo: $(tail -n 1 pagos_empresa.sh)"
106 | echo ""
107 |
108 | echo "-> 4.4 LA SOLUCION: El comprobador detecta que el Hash cambio y bloquea la ejecucion:"
109 | if sha256sum -c sumacomp.txt; then
110 |     ./pagos_empresa.sh
111 | else
112 |     echo "[BLOQUEADO]: La huella SHA-256 NO coincide. El archivo fue modificado y su ejecucion se
cancelo."
113 | fi
114 | echo ""
115 |
116 | echo "-> 4.5 Restaurando el archivo original y limpiando el entorno:"
117 | # Quitamos la linea maliciosa con 'sed', borramos sumacomp.txt y volvemos a dejar abierto el puerto
en servera
118 | sed -i '/VIRUS/d' pagos_empresa.sh
119 | rm -f sumacomp.txt
120 | ssh student@servera "echo student | sudo -S firewall-cmd --add-port=${PUERTO}/tcp >/dev/null 2>&1"
2>/dev/null
121 | echo "[OK]: Archivo pagos_empresa.sh restaurado y entorno limpio."
```

Desglose bloque por bloque

- **1. Conectividad y Firewall entre servidores (`servera` y `serverb`):**
  - **`ping -c 2 servera`** y **`ping -c 2 serverb`**: envían 2 paquetes a cada servidor para comprobar que ambos responden por red.
  - **`ssh student@servera "... firewall-cmd --remove-port=8080/tcp ..."`**: se conecta por SSH a `servera` y bloquea el puerto 8080 en su firewall.
  - **`ssh student@serverb "... firewall-cmd --add-port=8080/tcp ...; nohup python3 -m http.server 8080 ... &"`**: se conecta a `serverb`, abre el puerto 8080 en su firewall y levanta un servidor web en ese puerto.
  - **`curl -s --connect-timeout 2 http://servera:8080`** y **`http://serverb:8080`**: intenta conectarse a ambos desde afuera. `servera` rechaza la conexión (`[BLOQUEADO]`) y `serverb` la permite (`[PERMITIDO]`).
- **2. Creación del usuario `auditor` y contraseña con Hash + Salt en `/etc/shadow`:**

- `sudo useradd -m auditor`: crea el usuario `auditor` junto con su carpeta personal (`/home/auditor`).
- `echo "auditor:RedHat2026" | sudo chpasswd`: le asigna la contraseña `RedHat2026`.
- `sudo grep "^auditor:" /etc/shadow`: muestra la línea del usuario en `/etc/shadow`, comprobando que Linux nunca guarda la contraseña en texto plano sino como un resumen irreversible (**Hash + Salt**).

- **3. Lista de Control de Acceso (ACL) para el usuario `auditor`:**
  - `chmod 600 $ARCHIVO`: cierra el reporte confidencial para que solo su dueño tenga permiso; el grupo y el resto quedan sin acceso.
  - `setfacl -m u:auditor:r-- $ARCHIVO`: agrega una regla ACL para darle permiso de solo lectura (`r--`) únicamente al usuario `auditor`, sin abrir el archivo a los demás usuarios.
  - `ls -l $ARCHIVO`, `getfacl $ARCHIVO` y `sudo su - auditor -c "cat $ARCHIVO"`: muestran el signo `+` de la ACL activa, el detalle de permisos y comprueban que `auditor` puede leer el reporte.
- **4. Integridad con Hash SHA-256 sobre `pagos_empresa.sh`:**
  - `chmod 750 pagos_empresa.sh` y `sha256sum pagos_empresa.sh > sumacomp.txt`: restringe los permisos del script de pagos y guarda su huella digital oficial (SHA-256) en `sumacomp.txt`.
  - `if sha256sum -c sumacomp.txt; then ./pagos_empresa.sh; fi`: verifica la huella antes de ejecutar. Como el archivo está sano, coincide (`OK`) y se realiza la transferencia.
  - `echo 'echo "[VIRUS]: Robando contrasenas..."' >> pagos_empresa.sh`: simula el problema: un intruso agrega una línea maliciosa al final del archivo manteniendo el mismo nombre.
  - **Segunda verificación con `sha256sum -c sumacomp.txt`**: como el archivo cambió, la huella SHA-256 es completamente distinta (`FAILED`), entra por el bloque `else`, muestra `[BLOQUEADO]` y **cancela la ejecución** antes de que corra el virus.
  - `sed -i '/VIRUS/d' pagos_empresa.sh` y `rm -f sumacomp.txt`: restauran el script original y limpian el entorno.

Salida en consola al ejecutar el ejercicio:

```
----------------------------------------------------------------------
Seleccione el numero de unidad a ejecutar [1-6]: 6

======================================================================
 1. CONECTIVIDAD Y FIREWALL ENTRE SERVIDORES ('servera' Y 'serverb')
======================================================================
-> Probando conexion de red con servera y serverb:
PING servera.lab.example.com (172.25.250.10) 56(84) bytes of data.
64 bytes from servera.lab.example.com (172.25.250.10): icmp_seq=1 ttl=64 time=0.906 ms
64 bytes from servera.lab.example.com (172.25.250.10): icmp_seq=2 ttl=64 time=0.442 ms

--- servera.lab.example.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.442/0.674/0.906/0.232 ms

PING serverb.lab.example.com (172.25.250.11) 56(84) bytes of data.
64 bytes from serverb.lab.example.com (172.25.250.11): icmp_seq=1 ttl=64 time=0.625 ms
64 bytes from serverb.lab.example.com (172.25.250.11): icmp_seq=2 ttl=64 time=0.467 ms

--- serverb.lab.example.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 0.467/0.546/0.625/0.079 ms

-> [servera]: Bloqueando el puerto 8080/tcp en el Firewall...

-> [serverb]: Abriendo el puerto 8080/tcp en el Firewall y levantando servicio...
8080/tcp

--- PRUEBA DE CONEXION AL PUERTO 8080 ---
1) Intentando conectar a servera:8080 (Puerto BLOQUEADO por Firewall):
   [BLOQUEADO]: El Firewall de 'servera' rechazo la conexion al puerto 8080.
2) Intentando conectar a serverb:8080 (Puerto HABILITADO en Firewall):
   [PERMITIDO]: Conexion exitosa con 'serverb' en el puerto 8080.
```

```
========================================================================
 3. LISTA DE CONTROL DE ACCESO (ACL) PARA EL USUARIO 'auditor'
========================================================================
-> Otorgando permiso de lectura SOLO a 'auditor' mediante ACL (setfacl):
-rw-r-----+ 1 student student 36 Nov  1 21:38 /tmp/reporte_confidencial.txt
getfacl: Removing leading '/' from absolute path names
# file: tmp/reporte_confidencial.txt
# owner: student
# group: student
user::rw-
user:auditor:r--
group::---
mask::r--
other::---

-> Probando leer el archivo con el usuario 'auditor':
DATOS SECRETOS DEL SERVIDOR RED HAT

========================================================================
 4. INTEGRIDAD CON HASH SHA-256 ANTE MODIFICACION DE 'pagos_empresa.sh'
========================================================================
-> 4.1 Guardamos la huella digital oficial (SHA-256) de pagos_empresa.sh sano:
2b4dddc81fd2e68a6c993dd7c9928aa52457b0996366abf3897ffee38421be3b  pagos_empresa.sh

-> 4.2 Ejecucion normal (verificando la huella antes de correr):
pagos_empresa.sh: OK
[AUTORIZADO]: La huella coincide. Ejecutando pagos_empresa.sh:
[SISTEMA DE PAGOS]: Conectando con el banco...
[SISTEMA DE PAGOS]: Transferencia de sueldos realizada con exito.

-> 4.3 EL PROBLEMA: Un intruso/virus agrega una linea maliciosa a pagos_empresa.sh:
Ultima linea agregada al archivo: echo "[VIRUS]: Robando contrasenas..."

-> 4.4 LA SOLUCION: El comprobador detecta que el Hash cambio y bloquea la ejecucion:
pagos_empresa.sh: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
[BLOQUEADO]: La huella SHA-256 NO coincide. El archivo fue modificado y su ejecucion se cancelo.

-> 4.5 Restaurando el archivo original y limpiando el entorno:
[OK]: Archivo pagos_empresa.sh restaurado y entorno limpio.
```

En resumen

Este único script recorre las cuatro capas de seguridad de la unidad: primero muestra en red cómo el firewall bloquea el puerto 8080 en `servera` y lo permite en `serverb`; segundo, crea al usuario `auditor` y verifica que su contraseña se guarda cifrada con Hash y Salt en `/etc/shadow`; tercero, usa una ACL para darle lectura exclusiva sobre un archivo cerrado con `chmod 600`; y cuarto, demuestra cómo un comprobador de integridad con `sha256sum` detecta que `pagos_empresa.sh` fue modificado por un virus y bloquea su ejecución.

Cierre de la Unidad 6

Implementamos una defensa en profundidad sobre Red Hat en cuatro niveles dentro de una misma demostración: en la red, con `firewall-cmd` controlamos qué puertos aceptan conexiones entre servidores (`servera` bloqueado y `serverb` habilitado); en la autenticación, comprobamos en `/etc/shadow` que las contraseñas se almacenan como Hash con Salt; en los permisos, aplicamos ACL (`setfacl`) para que únicamente el usuario `auditor` pueda leer un reporte confidencial; y en la integridad, verificamos la huella SHA-256 de `pagos_empresa.sh` antes de correrlo. Así, cuando el archivo es alterado con una línea maliciosa, la huella deja de coincidir y su ejecución se bloquea automáticamente.